LAPORAN PRATIKUM TEKNOLOGI IOT 2

Modul 4: Komunikasi Asinkron dan WebSockets

Dibuat oleh :

Nama : Gohan Kristian Pangaribuan

Kelas : TRI 3B

Npm:25782040

Dosen pengampu : Dr.Ir. Septafiansyah Dwi Putra, S.T.,M.T.

Agus ambarwari, S.Pd., M.Kom.

JURUSAN TEKNOLOGI INFORMASI

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

POLITEKNIK NEGERI LAMPUNG TAHUN

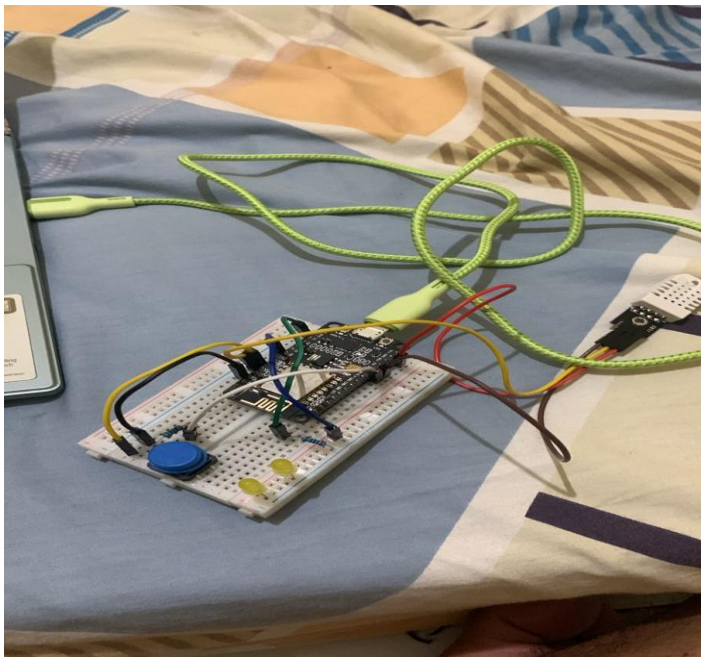
2025/2026

Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

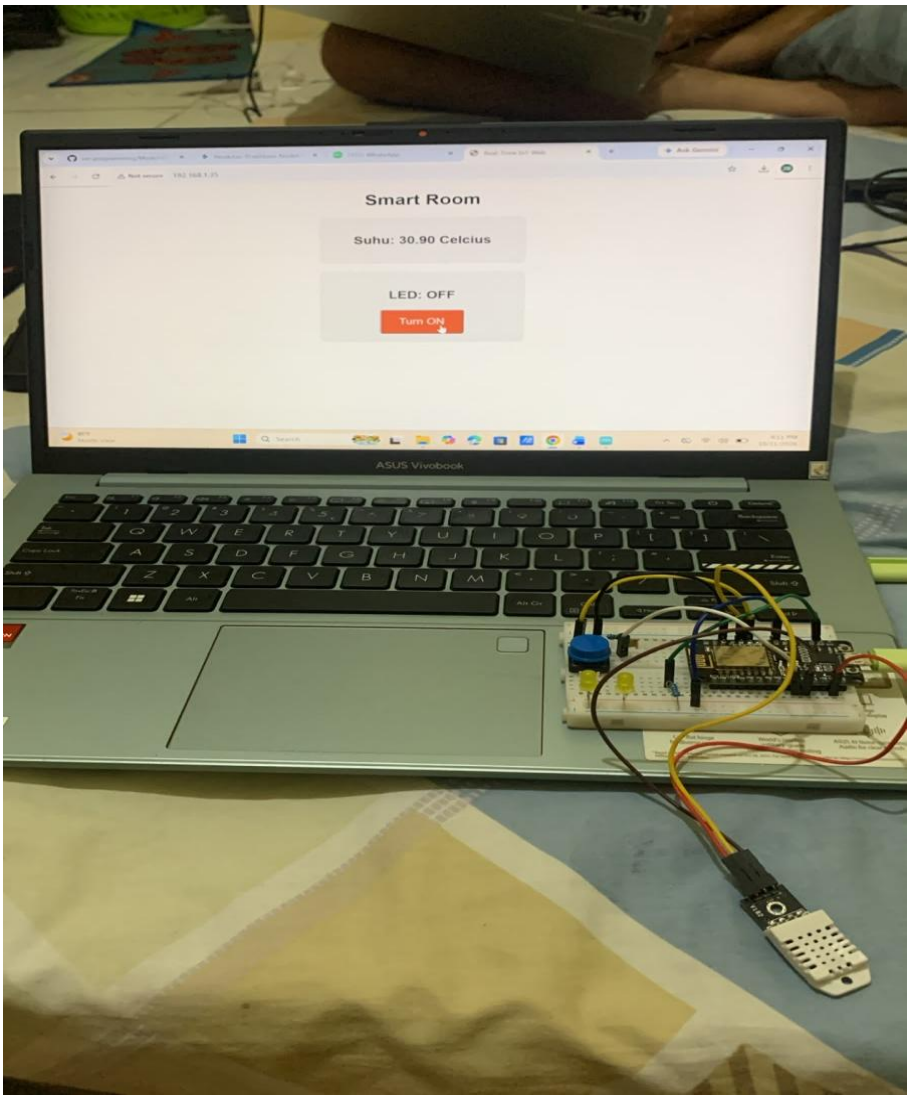
Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

Langkah Kerja Rangkaian:

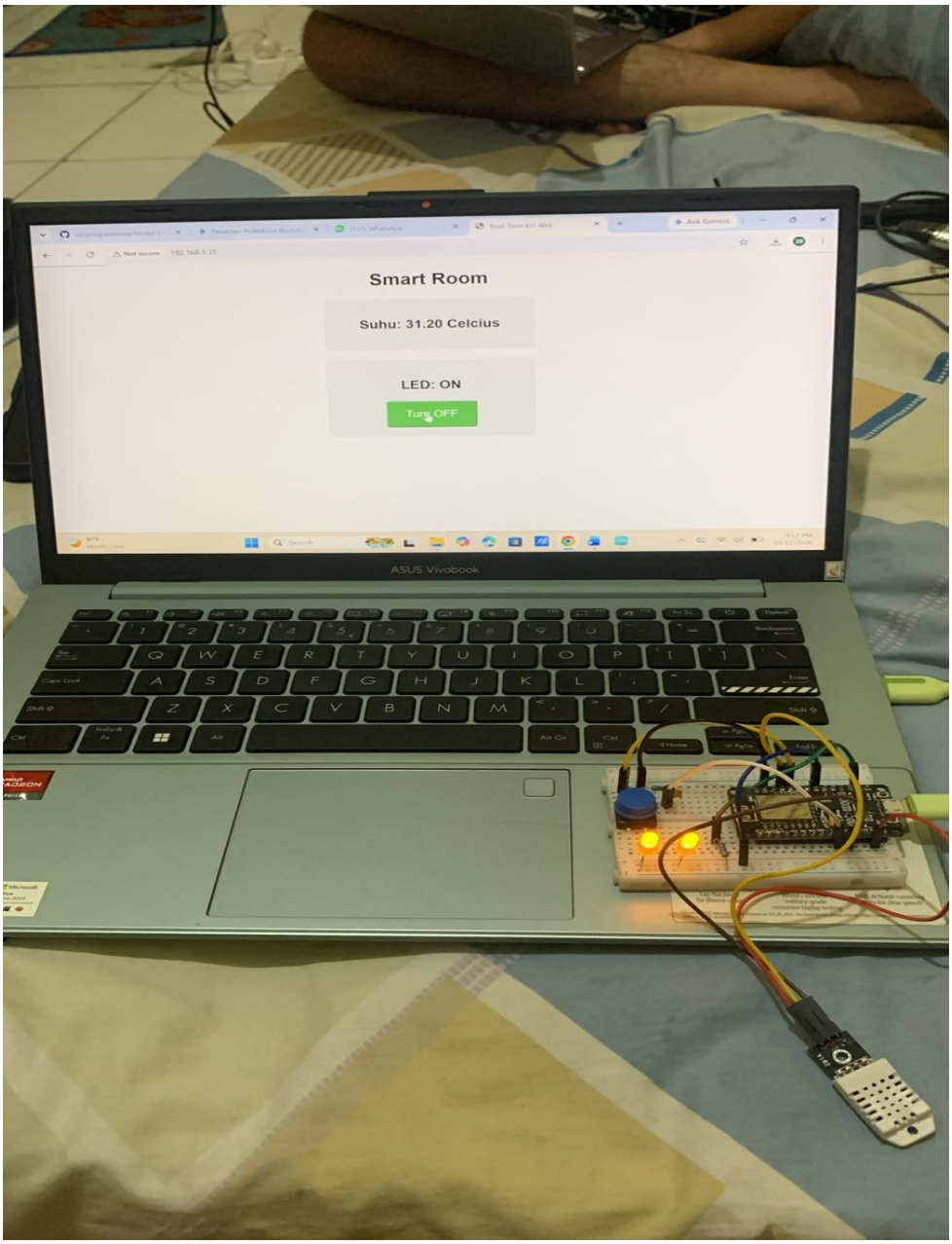
1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU *(sama seperti rangkaian LED pada Modul 3)*.
2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** *(konfigurasi resistor pull-down eksternal seperti pada Modul 1)*.



1.Kondisi LED jika kondisi lagi mati



2.Kondisi LED saat hidup :



4.6 Latihan:

Latihan 1: (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban : Efisiensi Bandwidth: WebSockets menggunakan koneksi persisten *full-duplex* yang hanya mengirimkan payload data mentah tanpa header HTTP berulang di setiap pesan, sedangkan Meta Refresh harus mengunduh ulang halaman utuh beserta seluruh header setiap kali *refresh*.

Tanpa Layar Berkedip: WebSockets memperbarui bagian halaman tertentu secara asinkron di latar belakang menggunakan pembaruan DOM parsial, sedangkan Meta Refresh melakukan *full page reload* yang memaksa browser merender ulang seluruh halaman sehingga terjadi kedipan.

Latihan 2 : (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

Jawaban : Memanggil fungsi delay(3000) di dalam void loop() NodeMCU akan membuat mikrokontroler berhenti merespons selama 3 detik, sehingga gagal memproses *ping* atau pesan WebSocket yang berakibat koneksi terputus (*timeout*), memicu Watchdog Timer (WDT Reset) yang membuat NodeMCU *restart* otomatis secara terus-menerus, serta menyebabkan antarmuka menjadi beku (*freezing*) karena data baru tidak bisa dikirim atau diterima secara *real-time*.

Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban : Target ID elemen HTML () yang umumnya menjadi target pengubahan teks pada skrip tersebut adalah statusHardware, ledStatus, atau btnText (sesuai dengan penamaan ID yang Anda definisikan di dalam file HTML praktikum) untuk memperbarui status perangkat atau tombol secara *real-time*.

Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

i. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar

menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).

ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawaban :

